```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Load the Digits Dataset
digits = load_digits()
X = digits.data
y = digits.target

# Standardizing the features (Essential for PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

def train_and_evaluate(X_train_data, X_test_data, title):
    """Function to train Logistic Regression and return accuracy"""
    model = LogisticRegression(max_iter=10000)
    model.fit(X_train_data, y_train)
    predictions = model.predict(X_test_data)
    accuracy = accuracy_score(y_test, predictions)
    print(f"Accuracy with {title}: {accuracy:.4f}")
    return accuracy

# --- Scenario 1: Original Features (No PCA) ---
acc_original = train_and_evaluate(X_train, X_test, "Original Features
(64 dimensions)")

# --- Scenario 2: PCA preserving 80% of Variance ---
pca_80 = PCA(n_components=0.80)
X_train_pca80 = pca_80.fit_transform(X_train)
X_test_pca80 = pca_80.transform(X_test)
acc_pca80 = train_and_evaluate(X_train_pca80, X_test_pca80, f"PCA 80%
Variance ({pca_80.n_components_} features)")

# --- Scenario 3: PCA with exactly 10 Features ---
pca_10 = PCA(n_components=10)
X_train_pca10 = pca_10.fit_transform(X_train)
X_test_pca10 = pca_10.transform(X_test)
acc_pca10 = train_and_evaluate(X_train_pca10, X_test_pca10, "PCA with
10 Features")

# --- Comparing Accuracy (Visualization) ---
methods = ['Original', 'PCA 80%', 'PCA 10 Feats']
```

```
accuracies = [acc_original, acc_pca80, acc_pca10]

plt.figure(figsize=(10, 6))
bars = plt.bar(methods, accuracies, color=['#3498db', '#2ecc71',
'#e67e22'])
plt.ylabel('Accuracy')
plt.title('Task: Accuracy Comparison (Original vs PCA)')
plt.ylim(0, 1.1)

# Adding data labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01,
f"{yval:.2%}", ha='center', va='bottom', fontweight='bold')

plt.show()
```